

What are Tensors? (In-depth Explanation)

1. Introduction: Why are we studying Tensors?

Before diving into "How" Machine Learning works, we must understand the data structure that makes it possible.

- **The "Why":** Tensors are the most basic data structure used by all leading Machine Learning and Deep Learning libraries (like Scikit-Learn, TensorFlow, PyTorch).
- **Fundamental Building Block:** Whether you are solving a simple ML problem or a complex Deep Learning task, you *must* deal with Tensors.
- **Real-world Impact:** The library "TensorFlow" (by Google) is named after this concept because "Tensors" "Flow" through the system to produce results.

2. What is a Tensor?

Definition: A Tensor is a **container for numbers**.

- It is a data structure used to store data (mostly numerical).
- While you can technically store characters/strings, 99.9% of the time in ML, Tensors contain numbers.
- **Relationship with Math/Physics:** The term "Tensor" comes from Mathematics and Physics. In Computer Science, you can think of them as **N-Dimensional Arrays**.

3. Types of Tensors (Based on Dimensions)

A. 0D Tensor (Scalar)

A single number is called a Scalar or a 0-Dimensional Tensor.

- **Example:** 5, 42, 3.14
- **Dimension:** 0 (It has no axes).

Code Implementation (NumPy):

```
import numpy as np
```

```
# Creating a 0D Tensor (Scalar)
a = np.array(4)
```

```
# Checking the number of dimensions
print(a.ndim)
```

Line-by-line Explanation:

1. `import numpy as np`: We import the NumPy library, which is the standard tool for handling arrays/tensors in Python.
2. `a = np.array(4)`: We create a NumPy array containing just one number (4).
3. `print(a.ndim)`: `.ndim` is a property that tells us the "rank" or "number of dimensions" of the tensor. For a scalar, this will output 0.

B. 1D Tensor (Vector)

An array of numbers (a list) is called a Vector or a 1-Dimensional Tensor.

- **Example:** `[1, 2, 3, 4]`
- **Dimension:** 1 (It has one axis).

Code Implementation:

```
import numpy as np

# Creating a 1D Tensor (Vector)
arr = np.array([1, 2, 3, 4])

# Checking the number of dimensions
print(arr.ndim)
```

Line-by-line Explanation:

1. `arr = np.array([1, 2, 3, 4])`: We pass a Python list to `np.array()` to create a 1D structure.
2. `print(arr.ndim)`: This will output 1.

★ CRITICAL DISTINCTION (Vector Dimension vs. Tensor Dimension):

A 1D Tensor is a **Vector**. However, a vector has its own "dimensions" based on how many numbers are inside it.

- `[1, 2]` is a **2D Vector** but still a **1D Tensor**.
- `[1, 2, 3, 4]` is a **4D Vector** but still a **1D Tensor**.
- *Tip:* In ML, "1D Tensor" tells you about the *structure* (a list), while "4D Vector" tells you about the *data points* inside that list.

C. 2D Tensor (Matrix)

A collection of vectors arranged in rows and columns is called a Matrix or a 2-Dimensional Tensor.

- **Example:** A spreadsheet or a table.
- **Dimension:** 2 (It has two axes: Rows and Columns).

Code Implementation:

```
import numpy as np

# Creating a 2D Tensor (Matrix)
mat = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Checking dimensions
print(mat.ndim)
```

Line-by-line Explanation:

1. `mat = np.array([[...]])`: We create a "list of lists". Each inner list represents a row.
2. `print(mat.ndim)`: This will output 2.

D. 3D Tensor

A collection of matrices is called a 3-Dimensional Tensor.

- **Visualizing it:** Think of it as a "cube" of numbers or a stack of matrices.
- **Axes:** It has three axes (Depth/Batch, Rows, Columns).

E. Higher Dimensional Tensors (4D, 5D, etc.)

- **4D Tensor:** A collection/vector of 3D Tensors.
- **5D Tensor:** A matrix (collection) of 3D Tensors.
- **The General Rule:** As you increase dimensions, you are simply "collecting" tensors of the previous dimension.
 - 1D = Collection of 0D
 - 2D = Collection of 1D
 - 3D = Collection of 2D... and so on.

4. Key Properties of Tensors

When working with any Tensor, you must know these three properties:

1. **Rank (ndim):** The number of axes in the tensor.
 - Scalar = Rank 0
 - Vector = Rank 1
 - Matrix = Rank 2
2. **Shape:** A tuple of integers that tells you how many items are in each axis.
 - Example: A matrix with 2 rows and 3 columns has a shape of (2, 3).
3. **Size:** The total number of elements in the tensor.
 - *Shortcut:* Multiply all the numbers in the **Shape**.
 - Example: Shape (2, 3) → Size = 2 * 3 = 6.

5. Practical Examples in Machine Learning

1D Tensor Example: Individual Record

In a dataset (e.g., student placement), a single student's data is a 1D Tensor.

- Student A: [CGPA: 8.5, IQ: 100, State: 1]
- This is a 1D Tensor (Vector).

2D Tensor Example: Full Dataset

A collection of multiple students' data forms a table.

- Row 1: Student A
- Row 2: Student B
- Row 3: Student C
- This is a 2D Tensor (Matrix).

3D Tensor Example: NLP & Time Series

1. **NLP (Natural Language Processing):**
 - A word is represented as a vector.
 - A sentence is a collection of word vectors (Matrix).
 - A batch of sentences is a collection of matrices → **3D Tensor**.
2. **Time Series (Stock Market):**
 - Daily data (High, Low) = 1D Vector.
 - One year of daily data = 2D Matrix (365 days x 2 features).
 - 10 years of data = Stack of 10 matrices → **3D Tensor** (10, 365, 2).

4D Tensor Example: Images

A color image has 3 "channels" (Red, Green, Blue).

- Each channel is a 2D Matrix of pixel values.
- One color image = 3 Matrices stacked → **3D Tensor** (3, Height, Width).
- A batch of 50 images = **4D Tensor** (50, 3, Height, Width).

5D Tensor Example: Video

A video is a sequence of images (frames) over time.

1. One Frame = 3D Tensor.
2. One Video (e.g., 60 seconds) = Collection of frames → 4D Tensor.
3. A batch of multiple videos = **5D Tensor**.

6. Important Reasoning: The "Memory" Problem (The Why)

The teacher explains why we don't usually go beyond 5D.

- **The Calculation:** * 4 videos, 60 seconds each, 30 FPS, 480p resolution, 3 color channels.
 - Total elements = $4 * (60 * 30) * 480 * 720 * 3$.
 - This results in a massive number of elements.
 - If each number takes 32 bits (float32), 4 minutes of video would require roughly **27 GB of RAM** to process as a raw tensor!
- **The Lesson:** This is why we use "Codecs" (MP4, MKV) to compress data. In raw ML processing, high-dimensional data is extremely memory-intensive.

Key Takeaways / Highlights

- **Highlight:** Rank = Number of Axes = ndim.
- **Tip:** If you see [...], look at the number of nested brackets to quickly identify the dimension.
- **Repetition:** The teacher emphasizes that you should not confuse "Vector Dimension" (internal elements) with "Tensor Dimension" (Rank).